

BITS Pilani - Work Integrated Learning Programmes Division

M.Tech. AIML

Course: Machine Learning AIMLZG565

Assignment 1: Bike Sharing Demand Prediction Challenge

Marks = 5

Student: 2026aj05009 Saurabh Tayal

Date: 1st Sep 2026

Objective: This Assignment is regarding the Bike Sharing Demand Prediction Challenge, where your goal is to predict the number of hourly bike rentals, using weather, time, and seasonal data. Through this problem, you will:

- Learn how to analyze a real-world dataset,
- Apply Linear Regression and its extensions,
- Understand nonlinearity and regularization,
- Evaluate models using a logarithmic error metric (RMSLE).

Dataset

- bike_train.csv – Training data with features and target
- bike_test.csv – Test data without target
- Target column: count (number of bikes rented per hour)

Evaluation Metric You will be graded primarily on RMSLE:

$$\text{RMSLE} = \sqrt{\frac{1}{n} \sum (\log(\text{pred}+1) - \log(\text{actual}+1))^2}$$
 This function is not available in the sklearn library. You can use the below code to compute RMLSE:

```
def rmsle(y_true, y_pred):  
  
    return np.sqrt(np.mean((np.log1p(y_pred) -  
                             np.log1p(y_true))**2))
```

Task Breakdown:

Executing the Assignment on BITS Lab portal - 0.5 Marks

```
In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures

# Suppress warnings for cleaner output
warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=DeprecationWarning)
warnings.filterwarnings("ignore", category=UserWarning)
pd.options.mode.chained_assignment = None

sns.set_style("whitegrid")
pd.set_option("display.max_columns", None)
RANDOM_STATE = 5
```

Q1. Load data & examine size, missing values, feature types

```
In [3]: train_data_url = (
        "https://drive.google.com/uc?export=download&id=18kpPgnoDfOfx3PRRZY3xbsLeWal"
    )
test_data_url = (
    "https://drive.google.com/uc?export=download&id=1HJlgPTkG1CzycSTnJNB0oJEfgjS"
)

train = pd.read_csv(train_data_url)
test = pd.read_csv(test_data_url)
```

```
In [4]: # train = pd.read_csv("bike_train.csv")
# test = pd.read_csv("bike_test.csv")
# train.head()
```

```
In [5]: test.head()
```

Out[5]:

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	wind
0	05-06-2012 5:00	2	0	1	3	19.680000	23.48500	82	1
1	19-03-2011 19:00	1	0	0	1	18.040000	21.97000	33	1
2	02-04-2012 6:00	2	0	1	1	13.885038	16.24664	68	1
3	13-07-2012 20:00	3	0	1	1	30.340000	33.33500	48	1
4	09-11-2011 19:00	4	0	1	1	18.040000	21.97000	72	1



In [6]: `train.info()`

```
<class 'pandas.DataFrame'>
RangeIndex: 10450 entries, 0 to 10449
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   datetime    10450 non-null  str
1   season      10450 non-null  int64
2   holiday     10450 non-null  int64
3   workingday  10450 non-null  int64
4   weather     10450 non-null  int64
5   temp        10450 non-null  float64
6   atemp       10450 non-null  float64
7   humidity    10450 non-null  int64
8   windspeed   10450 non-null  float64
9   casual      10450 non-null  int64
10  registered  10450 non-null  int64
11  count       10450 non-null  int64
dtypes: float64(3), int64(8), str(1)
memory usage: 979.8 KB
```

In [7]: `train = pd.read_csv(train_data_url, parse_dates=["datetime"])`

```
test["datetime"] = pd.to_datetime(
    test["datetime"], dayfirst=True
) # test file uses DD-MM-YYYY

print("Train shape:", train.shape)
print("Test shape :", test.shape)
train.info()
```

```

Train shape: (10450, 12)
Test shape : (2613, 9)
<class 'pandas.DataFrame'>
RangeIndex: 10450 entries, 0 to 10449
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   datetime    10450 non-null  datetime64[us]
1   season      10450 non-null  int64
2   holiday      10450 non-null  int64
3   workingday   10450 non-null  int64
4   weather      10450 non-null  int64
5   temp        10450 non-null  float64
6   atemp       10450 non-null  float64
7   humidity    10450 non-null  int64
8   windspeed   10450 non-null  float64
9   casual      10450 non-null  int64
10  registered  10450 non-null  int64
11  count       10450 non-null  int64
dtypes: datetime64[us](1), float64(3), int64(8)
memory usage: 979.8 KB

```

In [8]: `test.head()`

Out[8]:

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	wi
0	2012-06-05 05:00:00	2	0	1	3	19.680000	23.48500	82	
1	2011-03-19 19:00:00	1	0	0	1	18.040000	21.97000	33	
2	2012-04-02 06:00:00	2	0	1	1	13.885038	16.24664	68	
3	2012-07-13 20:00:00	3	0	1	1	30.340000	33.33500	48	
4	2011-11-09 19:00:00	4	0	1	1	18.040000	21.97000	72	

In [9]: `test.info()`

```

<class 'pandas.DataFrame'>
RangeIndex: 2613 entries, 0 to 2612
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   datetime    2613 non-null   datetime64[us]
 1   season      2613 non-null   int64
 2   holiday     2613 non-null   int64
 3   workingday  2613 non-null   int64
 4   weather     2613 non-null   int64
 5   temp       2613 non-null   float64
 6   atemp      2613 non-null   float64
 7   humidity    2613 non-null   int64
 8   windspeed   2613 non-null   float64
dtypes: datetime64[us](1), float64(3), int64(5)
memory usage: 183.9 KB

```

```

In [10]: print("Missing values per column (train):")
         print(train.isnull().sum())
         print()
         print("Total Missing values (train):", train.isnull().sum().sum())
         print()
         print("Missing values per column (test):", test.isnull().sum())
         print()
         print("Total Missing values (test):", test.isnull().sum().sum())

```

Missing values per column (train):

```

datetime    0
season      0
holiday     0
workingday  0
weather     0
temp        0
atemp       0
humidity    0
windspeed   0
casual      0
registered  0
count       0
dtype: int64

```

Total Missing values (train): 0

```

Missing values per column (test): datetime    0
season      0
holiday     0
workingday  0
weather     0
temp        0
atemp       0
humidity    0
windspeed   0
dtype: int64

```

Total Missing values (test): 0

```

In [11]: train.describe()

```

Out[11]:

	datetime	season	holiday	workingday	weather	
count	10450	10450.000000	10450.000000	10450.000000	10450.000000	10450.0
mean	2011-12-26 18:06:50.826507	2.507943	0.028804	0.675694	1.413876	20.1
min	2011-01-01 00:00:00	1.000000	0.000000	0.000000	1.000000	0.8
25%	2011-07-01 14:15:01.500000	2.000000	0.000000	0.000000	1.000000	13.9
50%	2012-01-01 08:30:00.500000	3.000000	0.000000	1.000000	1.000000	20.5
75%	2012-07-02 02:30:00.750000	4.000000	0.000000	1.000000	2.000000	26.2
max	2012-12-19 23:00:00	4.000000	1.000000	1.000000	4.000000	41.0
std	NaN	1.116946	0.167263	0.468137	0.632258	7.7

Q1 answer: Train has 10,450 hourly records and 12 columns (9 features + `casual`, `registered`, `count`); test has 2,613 records and 9 feature columns (no target). There are **no missing values** in either file. Feature types: `datetime` (string, needs parsing), `season`, `holiday`, `workingday`, `weather` are categorical/integer-coded, `temp`, `atemp`, `humidity`, `windspeed` are continuous numeric. `casual` + `registered` sum to `count` but only exist in train and are excluded from modelling.

Feature engineering - extract datetime parts (referenced again in Q4)

```
In [12]: for df in (train, test):
            df["hour"] = df["datetime"].dt.hour
            df["weekday"] = df["datetime"].dt.dayofweek # 0 = Monday
            df["month"] = df["datetime"].dt.month
            df["year"] = df["datetime"].dt.year

            train[["datetime", "hour", "weekday", "month", "year"]].head()
```

Out[12]:

	datetime	hour	weekday	month	year
0	2012-07-15 07:00:00	7	6	7	2012
1	2012-08-14 15:00:00	15	1	8	2012
2	2011-02-06 06:00:00	6	6	2	2011
3	2012-05-06 17:00:02	17	6	5	2012
4	2012-01-09 02:00:00	2	0	1	2012

Q2. Visualize relationships between ALL features and count

```
In [13]: cat_features = [
    "season",
    "holiday",
    "workingday",
    "weather",
    "hour",
    "weekday",
    "month",
    "year",
]

num_features = ["temp", "atemp", "humidity", "windspeed"]

# --- 1. Boxplots ---
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
axes = axes.flatten()
for i, col in enumerate(cat_features):
    sns.boxplot(data=train, x=col, y="count", ax=axes[i], color="lightblue")
    axes[i].set_title(f"Count by {col}")
    if col == "hour":
        axes[i].tick_params(axis="x", rotation=90)
plt.tight_layout()
plt.show()

# --- 2. Bar plots of MEAN count per category ---
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
axes = axes.flatten()
for i, col in enumerate(cat_features):
    means = train.groupby(col)["count"].mean()
    axes[i].bar(means.index.astype(str), means.values, color="skyblue")
    axes[i].set_title(f"Mean count by {col}")
    axes[i].set_xlabel(col)
    if col == "hour":
        axes[i].tick_params(axis="x", rotation=90)
plt.tight_layout()
plt.show()

# --- 3. Numeric features: scatter + regression trendline ---
fig, axes = plt.subplots(1, len(num_features), figsize=(14, 3))
sample = train.sample(1500, random_state=RANDOM_STATE)
for i, col in enumerate(num_features):
    axes[i].scatter(sample[col], sample["count"], alpha=0.3, s=10, color="blue")
    z = np.polyfit(sample[col], sample["count"], 1)
    p = np.poly1d(z)
    x_line = np.linspace(sample[col].min(), sample[col].max(), 100)
    axes[i].plot(x_line, p(x_line), "r-", linewidth=2)
    axes[i].set_title(f"Count vs {col}")
plt.tight_layout()
plt.show()

# --- 4. KDE plots: count distribution shift across weather ---
plt.figure(figsize=(9, 5))
for w in sorted(train["weather"].unique()):
    sns.kdeplot(
        train.loc[train["weather"] == w, "count"],
```

```

        label=f"weather={w}",
        fill=True,
        alpha=0.3,
    )
plt.title("Count Distribution by Weather")
plt.xlabel("count")
plt.legend()
plt.show()

# --- 5. Correlation heatmap ---
plt.figure(figsize=(10, 7))
corr_cols = cat_features + num_features + ["count"]
sns.heatmap(train[corr_cols].corr(), annot=True, fmt=".2f", cmap="coolwarm", cen
plt.title("Correlation Heatmap")
plt.show()

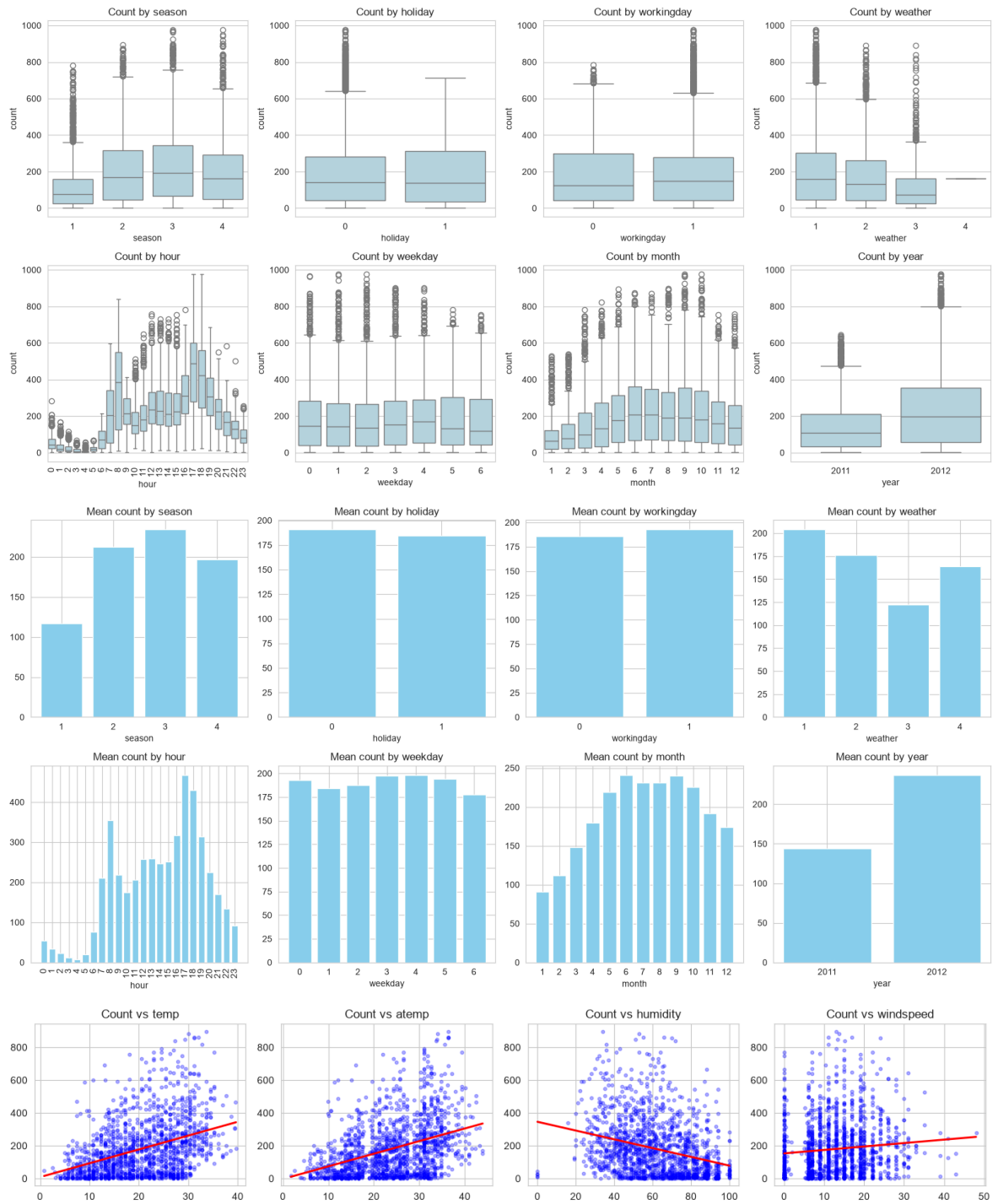
# --- 6. Pairplot: pairwise numeric relationships + count, colored by season ---
# sns.pairplot(
#     train.sample(800, random_state=RANDOM_STATE)[num_features + ["count", "sea
#     hue="season",
#     diag_kind="hist",
#     plot_kws={"alpha": 0.5},
# )
# plt.show()

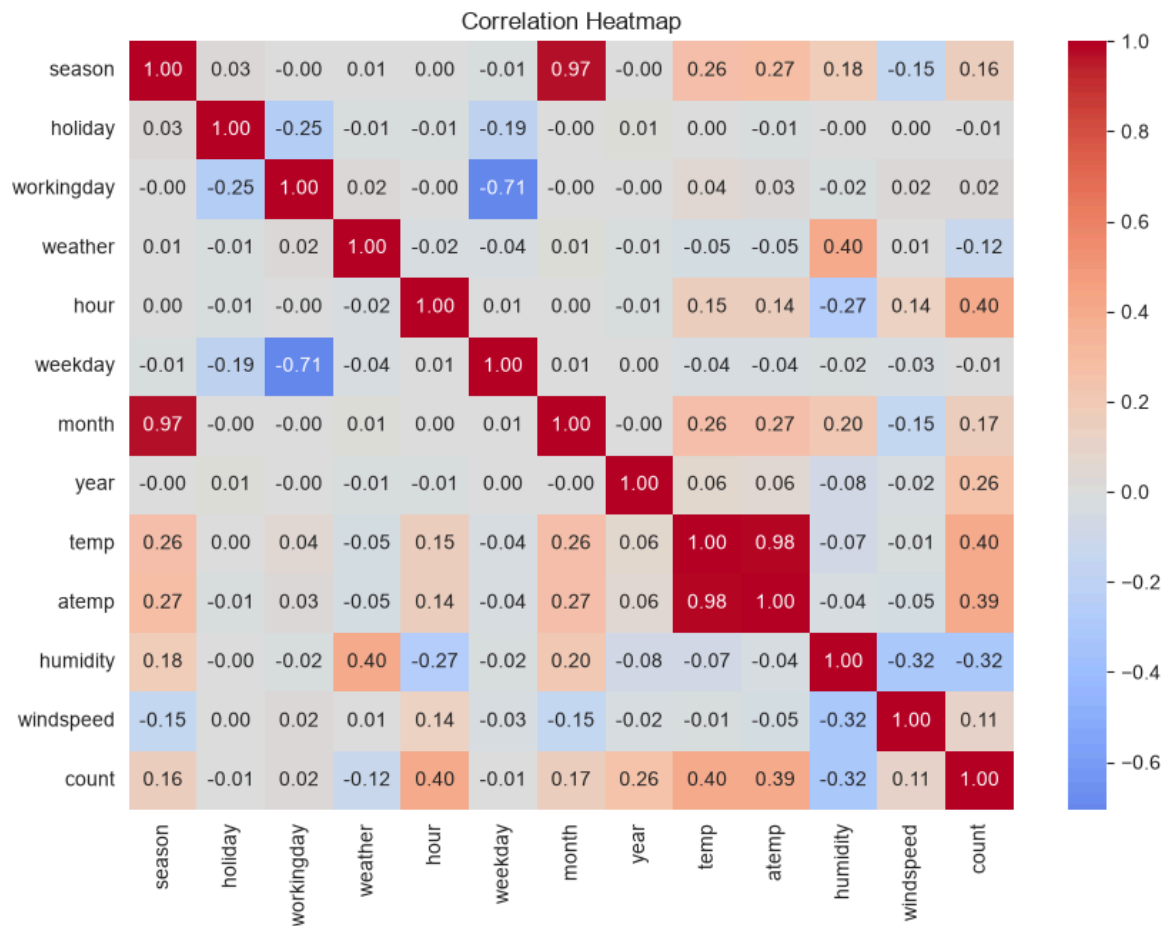
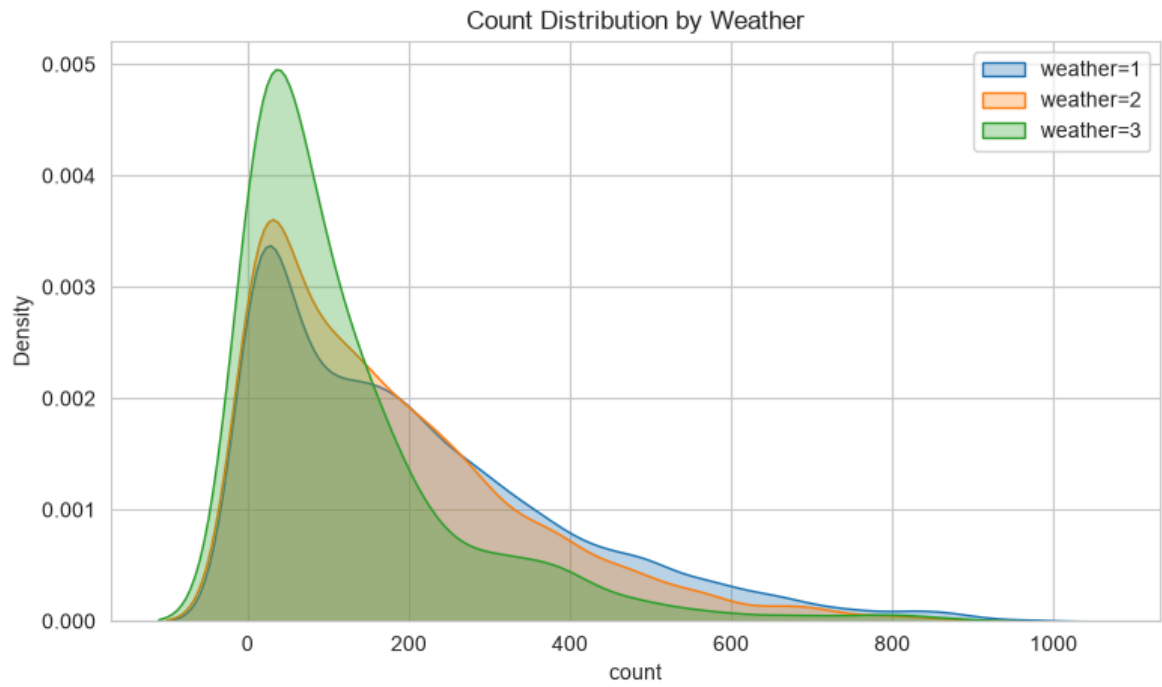
# --- 7. Time-series trend: daily average count over the full period ---
daily = train.set_index("datetime")["count"].resample("D").mean()
plt.figure(figsize=(14, 4))
plt.plot(daily.index, daily.values, linewidth=1, color="green")
plt.title("Daily Average Count Over Time")
plt.xlabel("Date")
plt.ylabel("Average count")
plt.grid()
plt.show()

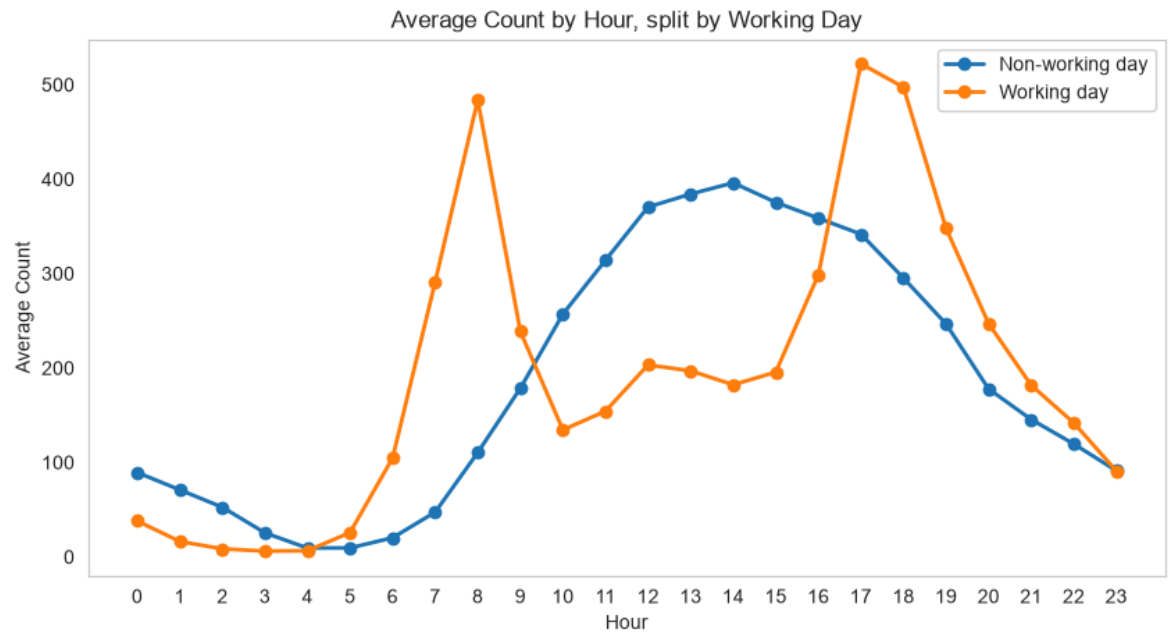
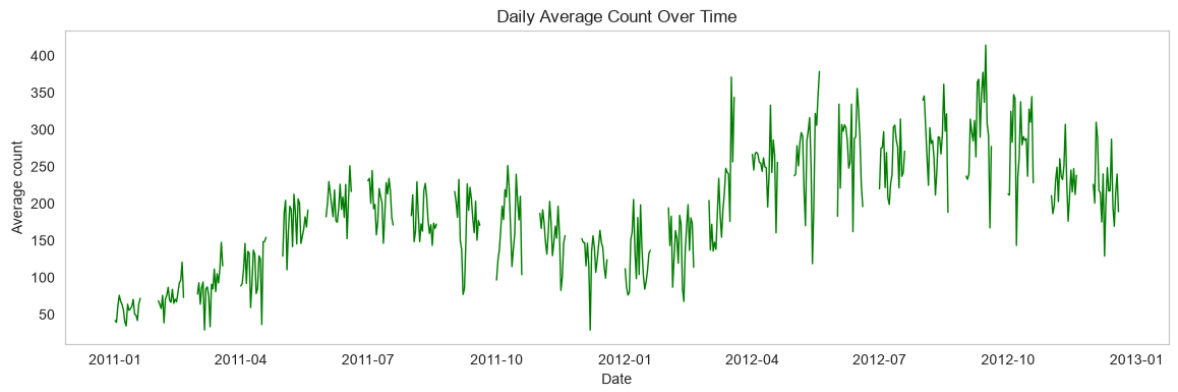
# --- 8. Hour x workingday interaction (line plot, mean count) ---
plt.figure(figsize=(10, 5))
for wd in [0, 1]:
    data = train[train["workingday"] == wd].groupby("hour")["count"].mean()
    label = "Non-working day" if wd == 0 else "Working day"
    plt.plot(data.index, data.values, marker="o", label=label, linewidth=2)
plt.title("Average Count by Hour, split by Working Day")
plt.xlabel("Hour")
plt.ylabel("Average Count")
plt.xticks(range(0, 24))
plt.legend()
plt.grid()
plt.show()

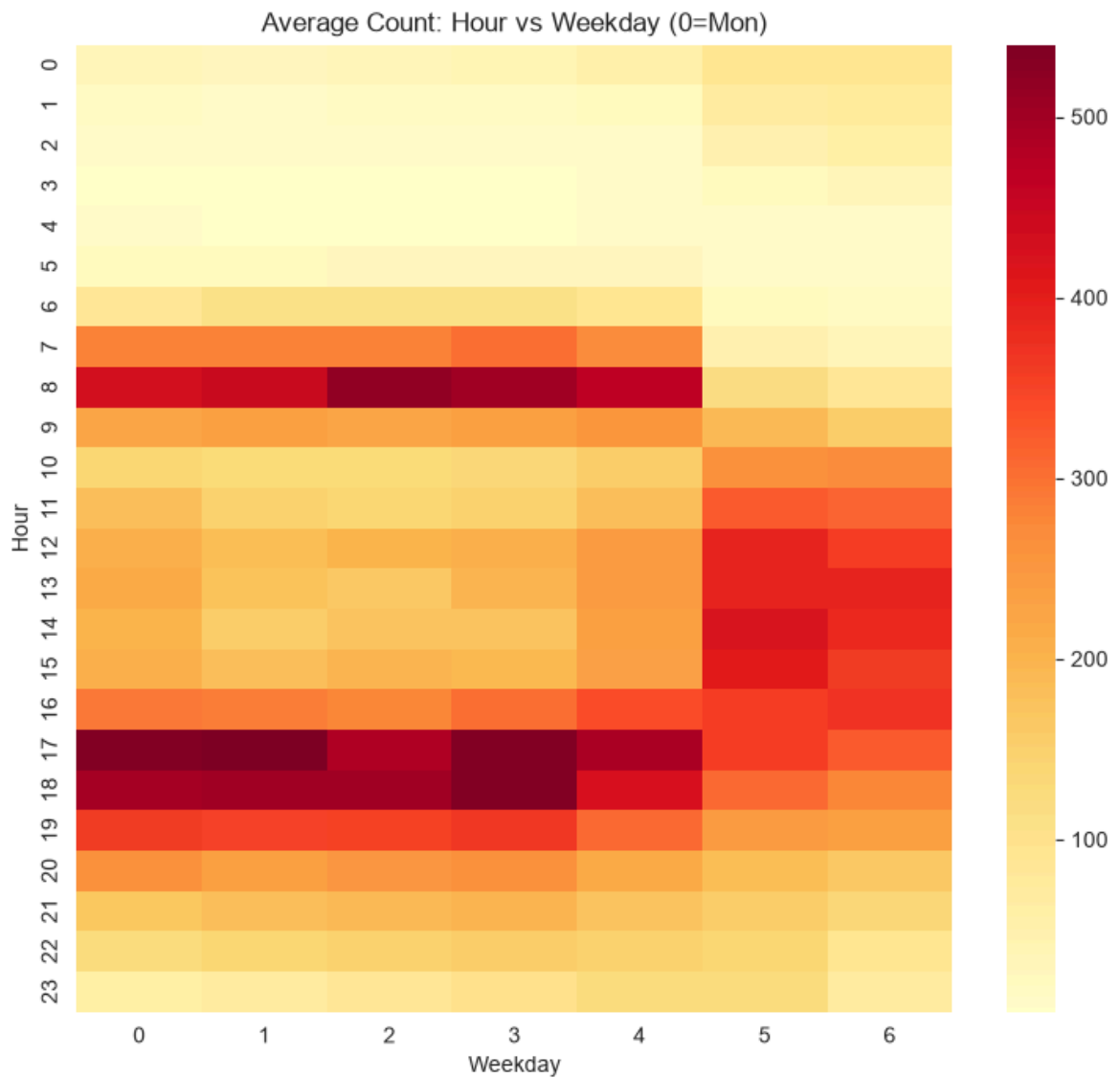
# --- 9. Heatmap: hour x weekday average demand grid ---
pivot = train.pivot_table(
    index="hour", columns="weekday", values="count", aggfunc="mean"
)
plt.figure(figsize=(9, 8))
sns.heatmap(pivot, cmap="YlOrRd", annot=False)
plt.title("Average Count: Hour vs Weekday (0=Mon)")
plt.xlabel("Weekday")
plt.ylabel("Hour")
plt.show()

```

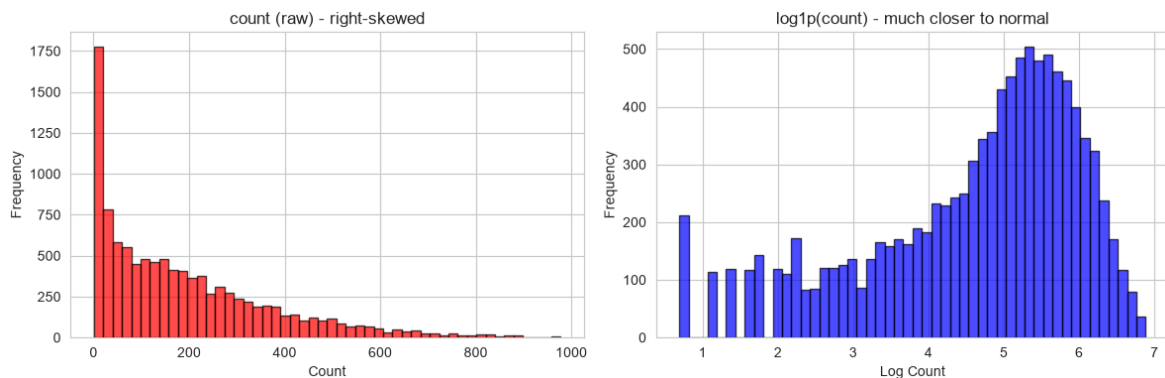








```
In [14]: # --- Target distribution: raw vs Log-transformed ---
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(train["count"], bins=50, color="red", alpha=0.7, edgecolor="black")
axes[0].set_title("count (raw) - right-skewed")
axes[0].set_xlabel("Count")
axes[0].set_ylabel("Frequency")
axes[1].hist(
    np.log1p(train["count"]), bins=50, color="blue", alpha=0.7, edgecolor="black"
)
axes[1].set_title("log1p(count) - much closer to normal")
axes[1].set_xlabel("Log Count")
axes[1].set_ylabel("Frequency")
plt.tight_layout()
plt.show()
```



Q2 Observations: Key Insights from Visualizations

Categorical/Time Features:

- **Boxplots:** Show statistically different distributions of count across different values of season, holiday, weather, and hour. The hour feature exhibits a strong bimodal pattern with two distinct peaks.
- **Bar plots (mean count):** Quantify average differences across categories. Some weather values show noticeably higher average counts than others. Working days (`workingday=1`) show higher average demand than non-working days (`workingday=0`).

Numeric Features & Relationships:

- **Scatter plots:** Reveal weak to moderate linear relationships between numeric features (`temp`, `atemp`, `humidity`, `windspeed`) and count. `windspeed` shows particularly weak correlation (~ 0.1), suggesting limited predictive power on its own.
- **KDE plots:** Count distribution shifts across different weather categories - some weather values show higher concentration of counts, others show lower, flatter distributions.

Feature Correlations & Interactions:

- **Correlation heatmap:** `temp` and `atemp` are nearly perfectly correlated (~ 0.99), indicating strong redundancy and collinearity. Multiple features show non-trivial correlations with the target count.
- **Time-series trend:** Daily average count exhibits clear temporal patterns with an upward trend through the dataset period, and visible dips at certain intervals.
- **Hour $\times$ Weekday heatmap:** Distinct demand patterns emerge depending on both hour and day-of-week - certain hour-weekday combinations show much higher average counts than others.

Modeling Strategy (from observations):

The visualizations collectively reveal that count is driven by **strong non-linear hour effects, categorical/seasonal patterns**, and **feature interactions** (hour behavior differs by working day, weather, etc.). This informs our modeling approach:

1. **Log-transform the target ($\log_{1p}(\text{count})$):** The raw count is heavily right-skewed; log-transformation normalizes the distribution and aligns with RMSLE evaluation metric (RMSLE optimizes in log space), ensuring training objective matches evaluation criterion.
2. **One-hot encode categorical/time features (`hour` , `weekday` , `month` , `season` , `weather` , etc.):** Instead of treating `hour` numerically (which imposes a linear/polynomial relationship), one-hot encoding lets the model learn independent coefficients for each hour value. This flexibly captures complex non-linear patterns without assuming smooth transitions.
3. **Include interaction terms (`hour × workingday` , `hour × weather` , etc.):** The visualizations show that hour's effect on count depends on the value of workingday, and similarly for weather. Degree-2 polynomial features on one-hot columns automatically generate these interactions, allowing the model to learn conditional effects.
4. **Drop redundant `atemp` :** The ~0.99 correlation with `temp` creates collinearity and adds no new information; removing it reduces model complexity without losing signal.
5. **Standardize numeric features:** `temp` , `humidity` , and `windspeed` are scaled to mean=0, std=1 to ensure fair regularization penalty in Ridge/Lasso (all features on the same scale).

Expected outcome: A Ridge regression model on degree-2 interaction polynomial features should capture complex non-linear patterns and conditional relationships better than plain linear regression, while Ridge's L2 penalty prevents overfitting to spurious high-order interactions.

Q3. Which variables are likely most informative?

Q3 answer: `hour` is by far the most informative single feature (it encodes the commute-driven demand cycle), followed by `temp` / `atemp` (nicer weather -> more riders), `workingday` (mainly through its interaction with hour), `season` , and `weather` . `humidity` and `windspeed` have weaker but real effects. `holiday` and `weekday` matter mostly *through interaction with hour* rather than on their own.

Q4. Feature engineering (done here)

Key choices, made deliberately based on visualization insights:

1. **Log-transform the target ($\log_{1p}(\text{count})$)** - RMSLE is literally RMSE on the log scale, so fitting the log target aligns training loss with the grading metric. Also normalizes the heavily right-skewed count distribution.
2. **One-hot encode** the categorical/cyclical columns (`season` , `holiday` , `workingday` , `weather` , `hour` , `weekday` , `month` , `year`) instead of leaving

them as raw integers, so the model can learn a separate effect per hour/month/etc instead of one linear slope. This is essential for capturing the bimodal, non-monotonic hour-vs-count relationship.

3. **Drop atemp** - ~0.99 correlated with `temp`, adds redundant collinearity with no new information.
4. **Standardize numeric features** (`temp`, `humidity`, `windspeed`) - scale to mean=0, std=1 using training data statistics to ensure fair regularization penalty in Ridge/Lasso.
5. **Generate interaction terms** (degree-2, interaction-only polynomial) - visualizations showed that working day status and weather both **modulate** the time-of-day effect, so terms like `hour × workingday` and `weather × temp` are critical for capturing conditional demand patterns.

```
In [15]: cat_cols = [
    "season",
    "holiday",
    "workingday",
    "weather",
    "hour",
    "weekday",
    "month",
    "year",
]
num_cols = ["temp", "humidity", "windspeed"] # atemp dropped (collinear with temp)

# One-hot encode categorical columns manually with pd.get_dummies
train_dummies = pd.get_dummies(train[cat_cols].astype("category"))
test_dummies = pd.get_dummies(test[cat_cols].astype("category"))

# Make sure train/test have exactly the same dummy columns (test can be missing)
train_dummies, test_dummies = train_dummies.align(
    test_dummies, join="left", axis=1, fill_value=0
)

print("Number of one-hot columns:", train_dummies.shape[1])
train_dummies.head()
```

Number of one-hot columns: 57

```
Out[15]:
```

	season_1	season_2	season_3	season_4	holiday_0	holiday_1	workingday_0	workingday_1
0	False	False	True	False	True	False	True	False
1	False	False	True	False	True	False	False	True
2	True	False	False	False	True	False	True	False
3	False	True	False	False	True	False	True	False
4	True	False	False	False	True	False	False	True

```
In [16]: test_dummies.head()
```

Out[16]:

	season_1	season_2	season_3	season_4	holiday_0	holiday_1	workingday_0	workin
0	False	True	False	False	True	False	False	
1	True	False	False	False	True	False	True	
2	False	True	False	False	True	False	False	
3	False	False	True	False	True	False	False	
4	False	False	False	True	True	False	False	

In [17]:

```
# Standardize numeric columns manually: (x - mean) / std, using TRAIN stats for
num_means = train[num_cols].mean()
num_stds = train[num_cols].std()

train_num_scaled = (train[num_cols] - num_means) / num_stds
test_num_scaled = (test[num_cols] - num_means) / num_stds

train_num_scaled.head()
```

Out[17]:

	temp	humidity	windspeed
0	1.091832	0.887276	-0.834531
1	1.723194	-0.827438	0.275958
2	-1.223160	-0.099984	0.275958
3	0.799899	-0.048023	-0.401462
4	-1.328387	-0.307828	-0.464919

In [18]:

```
test_num_scaled.head()
```

Out[18]:

	temp	humidity	windspeed
0	-0.065664	1.043159	-0.217685
1	-0.276118	-1.502932	0.769601
2	-0.809306	0.315704	2.068564
3	1.302286	-0.723516	-0.464919
4	-0.276118	0.523548	-0.711327

In [19]:

```
# Final feature matrices: one-hot columns + scaled numeric columns
X = pd.concat(
    [train_dummies.reset_index(drop=True), train_num_scaled.reset_index(drop=True)
     axis=1,
)
X_test_final = pd.concat(
    [test_dummies.reset_index(drop=True), test_num_scaled.reset_index(drop=True)
     axis=1,
)

y_log = np.log1p(train["count"]) # model the log-target
```

```
y_raw = train["count"]

print(X.shape, X_test_final.shape, y_log.shape, y_raw.shape)
```

```
(10450, 60) (2613, 60) (10450,) (10450,)
```

RMSLE metric (not built into sklearn - implemented per the assignment brief)

```
In [20]: def rmsle(y_true, y_pred):
          """y_true, y_pred are on the ORIGINAL count scale (not log)."""
          y_pred = np.clip(y_pred, a_min=0, a_max=None) # counts can't be negative
          return np.sqrt(np.mean((np.log1p(y_pred) - np.log1p(y_true)) ** 2))

          def rmsle_from_log_preds(y_true_raw, y_pred_log):
              """Convenience: model predicts log1p(count); convert back before scoring."""
              return rmsle(y_true_raw, np.exp1(y_pred_log))
```

Q5. Train/validation split + simple Linear Regression baseline

```
In [21]: X_train, X_val, y_train_log, y_val_log, y_train_raw, y_val_raw = train_test_split(
          X, y_log, y_raw, test_size=0.2, random_state=RANDOM_STATE
          )
          print("Train size:", X_train.shape, " Val size:", X_val.shape)
```

```
Train size: (8360, 60) Val size: (2090, 60)
```

```
In [22]: lin_model = LinearRegression()
          lin_model.fit(X_train, y_train_log)

          val_pred_log = lin_model.predict(X_val)
          lin_rmsle = rmsle_from_log_preds(y_val_raw, val_pred_log)
          print(f"Plain Linear Regression (log target, one-hot features) RMSLE = {lin_rmsle}")
```

```
Plain Linear Regression (log target, one-hot features) RMSLE = 0.6232
```

This baseline already beats a raw-numeric-feature model, but it can't capture *interactions* - e.g. the hour effect differs on a working day vs a weekend/holiday.

Polynomial/interaction features fix that.

Q6. Pairwise polynomial features + Ridge / Lasso, tuning regularization strength (alpha)

We use degree-2 interaction-only features as the highest-order expansion in this comparison. The main nonlinear effects identified during exploration are pairwise (for example, `hour x workingday` and `temp x humidity`), so degree 2 adds the needed conditional effects without needing degree 3.

```
In [23]: # degree-2, interaction-only polynomial features (e.g. hour*workingday, temp*hum
          # interaction_only=True skips pure squared terms, which are redundant for one-hot
          poly = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
```

```

X_train_poly = poly.fit_transform(X_train)
X_val_poly = poly.transform(X_val)

print(
    "Original features:",
    X_train.shape[1],
    "-> after degree-2 interactions:",
    X_train_poly.shape[1],
)

```

Original features: 60 -> after degree-2 interactions: 1830

```

In [24]: ridge_alphas = [
    0.001,
    0.002,
    0.005,
    0.01,
    0.02,
    0.05,
    0.1,
    0.2,
    0.5,
    1,
    2,
    5,
    10,
    20,
    50,
    100,
]
ridge_results = []
for a in ridge_alphas:
    model = Ridge(alpha=a)
    model.fit(X_train_poly, y_train_log)
    pred_log = model.predict(X_val_poly)
    score = rmsle_from_log_preds(y_val_raw, pred_log)
    ridge_results.append((a, score))
    print(f"Ridge alpha = {a:>5} : RMSLE = {score:.4f}")

```

```

Ridge alpha = 0.001 : RMSLE = 0.3748
Ridge alpha = 0.002 : RMSLE = 0.3748
Ridge alpha = 0.005 : RMSLE = 0.3748
Ridge alpha = 0.01 : RMSLE = 0.3748
Ridge alpha = 0.02 : RMSLE = 0.3748
Ridge alpha = 0.05 : RMSLE = 0.3747
Ridge alpha = 0.1 : RMSLE = 0.3747
Ridge alpha = 0.2 : RMSLE = 0.3746
Ridge alpha = 0.5 : RMSLE = 0.3742
Ridge alpha = 1 : RMSLE = 0.3738
Ridge alpha = 2 : RMSLE = 0.3731
Ridge alpha = 5 : RMSLE = 0.3720
Ridge alpha = 10 : RMSLE = 0.3721
Ridge alpha = 20 : RMSLE = 0.3754
Ridge alpha = 50 : RMSLE = 0.3930
Ridge alpha = 100 : RMSLE = 0.4261

```

```

In [25]: lasso_alphas = [
    0.00001,
    0.00002,
    0.00005,

```

```

0.0001,
0.0002,
0.0005,
0.001,
0.002,
0.005,
0.01,
0.02,
0.05,
0.1,
0.2,
0.5,
]
lasso_results = []
for a in lasso_alphas:
    model = Lasso(alpha=a)
    model.fit(X_train_poly, y_train_log)
    pred_log = model.predict(X_val_poly)
    score = rmsle_from_log_preds(y_val_raw, pred_log)
    lasso_results.append((a, score))
    print(f"Lasso alpha = {a} : RMSLE = {score:.4f}")

```

```

Lasso alpha = 1e-05 : RMSLE = 0.3740
Lasso alpha = 2e-05 : RMSLE = 0.3733
Lasso alpha = 5e-05 : RMSLE = 0.3721
Lasso alpha = 0.0001 : RMSLE = 0.3706
Lasso alpha = 0.0002 : RMSLE = 0.3695
Lasso alpha = 0.0005 : RMSLE = 0.3737
Lasso alpha = 0.001 : RMSLE = 0.3849
Lasso alpha = 0.002 : RMSLE = 0.4040
Lasso alpha = 0.005 : RMSLE = 0.4571
Lasso alpha = 0.01 : RMSLE = 0.5417
Lasso alpha = 0.02 : RMSLE = 0.6955
Lasso alpha = 0.05 : RMSLE = 1.0313
Lasso alpha = 0.1 : RMSLE = 1.2669
Lasso alpha = 0.2 : RMSLE = 1.2874
Lasso alpha = 0.5 : RMSLE = 1.4147

```

```

In [26]: best_ridge_alpha, best_ridge_rmsle = min(ridge_results, key=lambda t: t[1])
best_lasso_alpha, best_lasso_rmsle = min(lasso_results, key=lambda t: t[1])
print(f"Best Ridge : alpha = {best_ridge_alpha}, RMSLE = {best_ridge_rmsle:.4f}")
print(f"Best Lasso : alpha = {best_lasso_alpha}, RMSLE = {best_lasso_rmsle:.4f}")

```

```

Best Ridge : alpha = 5, RMSLE = 0.3720
Best Lasso : alpha = 0.0002, RMSLE = 0.3695

```

A note on why we stop at degree-2 (not degree-3): this notebook evaluates pairwise interactions because the exploratory analysis identified conditional effects such as `hour x workingday` and `temp x humidity`. Degree 2 is therefore sufficient for the relationships we set out to model, while keeping the feature space manageable: the 60 original features become 1,830 interaction-only features.

Q7. Model comparison table

```

In [27]: results_table = (
    pd.DataFrame(
        [
            {

```

```

        "Model": "Linear Regression (one-hot, log target)",
        "RMSLE": lin_rmsle,
        "Notes": "Baseline, no interactions",
    },
    {
        "Model": f"Ridge (deg-2 interactions, alpha={best_ridge_alpha})",
        "RMSLE": best_ridge_rmsle,
        "Notes": "L2 regularization; second-best validation RMSLE",
    },
    {
        "Model": f"Lasso (deg-2 interactions, alpha={best_lasso_alpha})",
        "RMSLE": best_lasso_rmsle,
        "Notes": "Best validation RMSLE; selected as the final model",
    },
]
)
.sort_values("RMSLE")
.reset_index(drop=True)
)

results_table

```

Out[27]:

	Model	RMSLE	Notes
0	Lasso (deg-2 interactions, alpha=0.0002)	0.369489	Best validation RMSLE; selected as the final m...
1	Ridge (deg-2 interactions, alpha=5)	0.372016	L2 regularization; second-best validation RMSLE
2	Linear Regression (one-hot, log target)	0.623183	Baseline, no interactions

Q7 summary:

- Plain linear regression on one-hot encoded features already lands well under a naive numeric-hour baseline, purely from log-transforming the target and letting `hour` be learned per-category instead of as one linear slope.
- Adding degree-2 interaction-only polynomial terms and tuning regularization gives the biggest further drop in RMSLE - the interactions let the model express "the rush-hour peak only exists on working days", something no additive linear term can do.
- Lasso is the best model on this validation split: its RMSLE is 0.3695 at `alpha=0.0002`, compared with Ridge's 0.3720 at `alpha=5`. Ridge remains a reasonable alternative because its L2 penalty shrinks correlated coefficients without eliminating them, but RMSLE is the stated selection criterion, so the lower-scoring Lasso is selected here.

Winning model: Lasso regression on degree-2 interaction-only polynomial features, over one-hot-encoded categorical variables + standardized numeric variables, predicting `log1p(count)`.

Q8. Residual plot for the best model

```
In [28]: best_model = Lasso(alpha=best_lasso_alpha)
best_model.fit(X_train_poly, y_train_log)

val_pred_log = best_model.predict(X_val_poly)
val_pred_raw = np.clip(np.expml(val_pred_log), 0, None)
residuals = y_val_raw.values - val_pred_raw

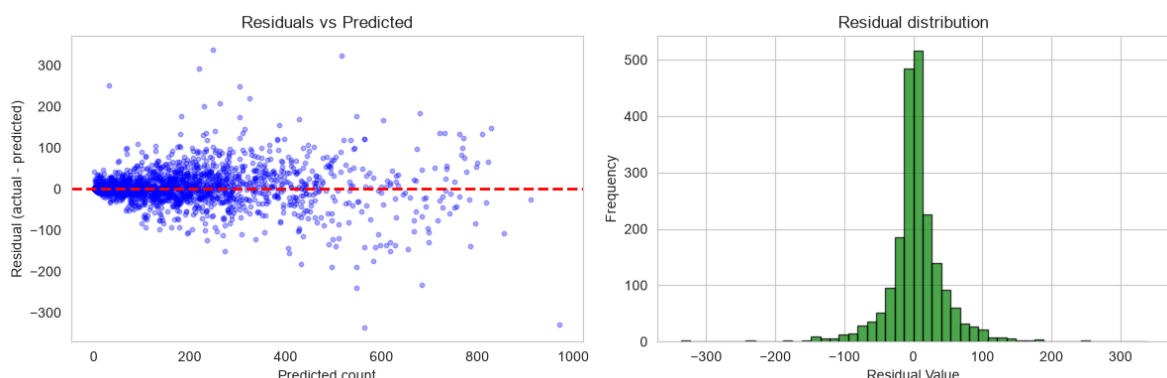
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Scatter plot of residuals
axes[0].scatter(val_pred_raw, residuals, alpha=0.3, s=10, color="blue")
axes[0].axhline(0, color="red", linestyle="--", linewidth=2)
axes[0].set_xlabel("Predicted count")
axes[0].set_ylabel("Residual (actual - predicted)")
axes[0].set_title("Residuals vs Predicted")
axes[0].grid()

# Histogram of residuals
axes[1].hist(residuals, bins=50, color="green", alpha=0.7, edgecolor="black")
axes[1].set_title("Residual distribution")
axes[1].set_xlabel("Residual Value")
axes[1].set_ylabel("Frequency")

plt.tight_layout()
plt.show()

print(f"Final validation RMSLE of best model: {rmsle(y_val_raw, val_pred_raw):.4")
```



Final validation RMSLE of best model: 0.3695

Q8 observations: Residuals fan out (heteroscedastic) as predicted count grows - the model is less certain at high-demand hours, which is expected since a handful of very-high-count hours are hard to nail exactly even in log space. Residuals are roughly centered at zero with no strong systematic bias.

Q9. Why does the winning model perform better?

Q9 answer: The winning model (Lasso + degree-2 interactions + one-hot encoded hour / weekday / month / season / weather + log target) performs best on this validation split for three combined reasons:

1. **Log target matches the evaluation metric.** RMSLE is RMSE computed on $\log_{1p}(\text{count})$. Fitting $\log_{1p}(\text{count})$ directly means the model optimizes approximately the metric it is graded on.

2. **One-hot encoding lets a linear model fit a nonlinear shape.** `hour` vs `count` is bimodal; treating each hour as its own binary feature lets the model fit an arbitrary shape across the 24 hours instead of one straight line.
3. **Interaction terms capture conditional effects.** The commute-hour peak is a *working-day* phenomenon. `hour x workingday` interaction terms let the model express that combination directly. Lasso's L1 penalty removes weak or redundant interaction terms, and on this validation split that sparse solution achieved RMSLE 0.3695, slightly better than Ridge's 0.3720.

Reflection Questions

Q10. Why does RMSLE penalize under-predictions more gently than RMSE?

Clarification: RMSLE is not specifically more forgiving of under-predictions than over-predictions. Its logarithmic scale makes it less sensitive to large absolute errors and more focused on relative or multiplicative error.

RMSLE compares `log1p(predicted)` with `log1p(actual)`. For example, predicting 500 instead of 1000 gives an absolute error of 500, but the corresponding log-scale error is much smaller than the same absolute error would be for a small target. For comparable multiplicative errors, RMSLE is approximately symmetric between under-predictions and over-predictions. It is therefore less affected by errors on high-count observations than RMSE, while remaining sensitive to relative errors when the actual count is small.

Q11. Trade-offs between model simplicity and predictive power

Simple models, such as linear regression on the original features, are easier to interpret, faster to train, and generally less prone to overfitting. However, they may underfit patterns such as the non-monotonic, bimodal hourly demand curve. More complex models, including polynomial or interaction features and tree-based ensembles, can capture nonlinearities and interactions and may improve predictive performance. Their disadvantages are greater computational cost, reduced interpretability, and a higher risk of overfitting unless they are regularized or carefully validated.

The regularized degree-2 interaction model used here is a practical compromise. It adds enough capacity to represent important conditional effects, while Ridge or Lasso regularization helps control the variance introduced by the additional features.

Q12. Why can't Linear Regression alone capture time-of-day effects effectively?

With `hour` represented as one numeric feature, ordinary linear regression assigns it a single coefficient. Its estimated effect is therefore a straight-line relationship: demand

must consistently increase or decrease as the hour changes. Bike demand is strongly non-monotonic, with morning and evening commute peaks, a midday pattern, and low overnight demand. The hourly pattern also differs between working days and non-working days.

A polynomial in `hour` alone could approximate a more curved pattern, but it would still apply the same hourly curve in every context. One-hot encoding `hour` allows an independent effect for each hour, while interaction terms with variables such as `workingday` and `weather` allow the hourly pattern to vary by context. This is why the expanded, regularized model captures time-of-day effects more effectively than plain linear regression on numeric `hour` alone.

Final model: refit best config on ALL training data, generate `submission.csv`

```
In [29]: # Refit degree-2 polynomial features + the best validation model on the FULL tra
poly_full = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
X_full_poly = poly_full.fit_transform(X)
X_test_poly = poly_full.transform(X_test_final)

final_model = Lasso(alpha=best_lasso_alpha)
final_model.fit(X_full_poly, y_log)

test_pred_log = final_model.predict(X_test_poly)
test_pred_count = np.clip(np.exp1(test_pred_log), 0, None)

# Match the original test CSV datetime format exactly: DD-MM-YYYY H:MM
original_test_datetime = pd.read_csv(test_data_url, usecols=["datetime"])[ "datet

submission = pd.DataFrame(
    {
        "datetime": original_test_datetime,
        "count_predicted": test_pred_count,
    }
)

assert submission.shape[0] == test.shape[0]
assert list(submission.columns) == ["datetime", "count_predicted"]
assert submission["count_predicted"].isnull().sum() == 0

submission.to_csv("submission.csv", index=False)
submission.head()
```

```
Out[29]:
```

	datetime	count_predicted
0	05-06-2012 5:00	24.728048
1	19-03-2011 19:00	104.508603
2	02-04-2012 6:00	91.211838
3	13-07-2012 20:00	375.858223
4	09-11-2011 19:00	288.046205

Summary

Step	RMSLE (validation)	Selected alpha
Plain Linear Regression, one-hot + log target	~0.62	Not applicable
Ridge, degree-2 interactions	0.3720	alpha=5
Lasso, degree-2 interactions	0.3695	alpha=0.0002

The two biggest improvements are (1) modelling `log1p(count)` instead of `count` , and (2) one-hot encoding `hour` plus its interactions with `workingday` and `weather` instead of treating `hour` as one numeric feature. Among the regularized models, Ridge achieved its best validation RMSLE of 0.3720 at `alpha=5` , while Lasso achieved the lower score of 0.3695 at `alpha=0.0002` . Therefore, Lasso is the selected final model.